

Operating Systems

A practical study reference with foundational concepts, examples, and common review questions.

1. Kernel and Processes

An operating system manages hardware and provides processes, virtual memory, files, and device access. The kernel handles privileged operations; applications request services through system calls. Interrupts notify the processor of events; context switches change the running execution state.

A process is a running program with an address space and resources. A thread is an execution path inside a process; threads share process memory. Processes provide stronger isolation but inter-process communication adds cost.

2. Scheduling and Synchronization

Schedulers select ready tasks. FCFS is simple; shortest-job-first can reduce average waiting under assumptions; round robin provides time slices; priority scheduling can starve low-priority work unless mitigated. I/O-blocked tasks wait for events and are not ready to run.

Race conditions depend on timing. Mutexes protect critical sections; semaphores count permits; condition variables wait for predicates. Deadlock requires mutual exclusion, hold-and-wait, no preemption, and circular wait. Systems prevent, avoid, detect, or recover from it.

3. Memory and Filesystems

Virtual memory gives a process an address space mapped to physical memory. Paging divides memory into pages and frames; a page fault triggers OS handling. A TLB caches translations. Thrashing occurs when excessive paging prevents useful work.

Filesystems organize persistent files and metadata. Journaling can improve crash consistency but is not a backup. Use permissions and isolation to protect data; test backup restoration.

4. Common Questions

A system call is a controlled request from user mode to the kernel. Virtual memory supports isolation and flexible memory management. A page fault can mean a page must be loaded or access permissions checked. A process has its own address space; threads commonly share one.